

Approach Document

Legal AI Assistant — Architecture, Design Decisions, and Honest Limitations

Contents

1. Overall Architecture
2. Design Decisions
3. Parser
4. OCR Fallback
5. Chunking Strategy
6. Embedding Model
7. Hybrid Search
8. MMR (Maximal Marginal Relevance)
9. Re-ranking
10. Citation Generation
11. Frontend
12. Backend
13. Deployment
14. Limitations
15. Future Improvements

1. Overall Architecture

The system is a citation-grounded legal Q&A; pipeline: PDF Upload -> Parsing -> OCR fallback -> Structure-aware Chunking -> Embeddings -> Hybrid Search (Vector + BM25) -> Reciprocal Rank Fusion -> MMR diversification -> Re-ranking + confidence filtering -> LLM generation -> Citation formatting -> Frontend delivery. See [Architecture_Diagram.pdf](#) for the visual.

The backend (FastAPI) is organized into a service layer (`app/services/`) behind thin API routes (`app/api/routes/`), with dependency injection used throughout so every stage - parser, chunker, embedder, vector store, keyword search, reranker, LLM provider - can be swapped or mocked without touching callers. This is what let every stage be tested in isolation with real (not mocked) logic, even where a real network-dependent backend (HuggingFace, OpenAI, a live Elasticsearch cluster) could not be reached from the development sandbox.

The frontend is a real Next.js 15 App Router application (not a static mockup), consuming the backend's REST and Server-Sent Events endpoints.

2. Design Decisions

Every major design choice in this project followed the same pattern: build the real thing, test it for real wherever possible, and when a real external dependency (a model download, a cloud service, a paid API) could not be reached from the development sandbox, build a genuine offline/local alternative rather than mocking the interface and calling it done. This is why, for example, both an embedded-Qdrant/BM25-local path AND a real-Qdrant/Elasticsearch path exist side by side - not as a compromise, but as two legitimate deployment sizes (see Limitations for exactly which paths were verified where).

Provider abstraction (embeddings, keyword search, reranking, LLM) is used consistently: one interface, multiple implementations, selected by a single environment variable. This is what makes the system's tech-stack flexibility (BGE/OpenAI embeddings, BM25/Elasticsearch keyword search, lightweight/cross-encoder reranking, Claude/GPT/Gemini generation) a real, working property of the code rather than a documentation claim.

3. Parser

PDF parsing (Milestone 1) uses PyMuPDF for native text extraction (fast, high fidelity for digital PDFs), with a per-page fallback to pdfplumber for table-heavy pages (PyMuPDF's plain-text mode flattens tables into unreadable text; pdfplumber preserves row/column structure). Every extracted page retains its exact page number, document identity, and a file hash for deduplication - since page-accurate citation is the entire point of the system, losing or misaligning page numbers at ingestion time cannot be fixed downstream no matter how good the LLM prompt is.

Document type (Act / Judgment / Tax Document / POV Document) can be user-tagged on upload or auto-suggested: a fast keyword-heuristic classifier runs first (regex signal scoring against the first ~2 pages), escalating to an LLM classification call only when heuristic confidence is low - a deliberate cost/latency tradeoff matching how a real production system would minimize unnecessary LLM calls.

4. OCR Fallback

Tesseract OCR is used only when a page's native text extraction yields near-empty output (configurable character threshold), rather than running OCR on every page regardless of need - OCR is roughly 10-50x slower than native extraction and introduces its own recognition errors, so it is reserved for genuinely scanned pages. Pages are rasterized at 300 DPI-equivalent zoom before OCR for reliable accuracy, and each page's metadata records exactly which extraction method (native / table-aware / OCR) produced its text plus an OCR confidence score, so evaluation can separately measure native-vs-OCR accuracy if needed.

This was verified with a genuinely constructed test case: a synthetic three-page PDF where the third page is a rasterized image with no text layer at all (not merely a low-quality scan simulation) - the parser correctly detected near-zero native text and routed it through Tesseract, recovering the page's content with a measured OCR confidence around 95%.

5. Chunking Strategy

Chunking (Milestone 2) is structure-aware rather than fixed-size: it splits on legal structural markers first (numbered paragraphs such as judgment paragraph numbers, and Section/Article/Clause/Chapter/Part headers for statutes), and only applies a size-cap sub-split - preferring sentence boundaries, with character overlap - to sections still too large after structural splitting. Small structural sections are merged with neighbors up to a minimum size, avoiding an index cluttered with near-empty fragments.

The concrete justification: a fixed-size chunker will happily cut 'Section 80G(5)(iv) requires X' from its qualifying clause 'provided that Y' into two separate chunks, so a retrieval that finds the requirement can miss its condition entirely. This project's test suite includes a dedicated test proving the real sample judgment's Section 80G(5)(iv) requirement and its registration-number condition stay together as one coherent chunk under this chunker - the exact failure mode structure-aware chunking exists to prevent.

Every chunk retains exact page provenance, including chunks that legitimately straddle a page boundary (tracked via per-page character-count spans), which is what lets the citation formatter (Milestone 7) report an accurate page range for any chunk regardless of where it falls relative to page breaks.

6. Embedding Model

Three embedding providers exist behind one interface: BAAI/bge-large-en-v1.5 (default - free, runs locally via sentence-transformers, no API key, asymmetric query/document convention handled internally), OpenAI text-embedding-3-large (API-based, no local compute, costs per token), and a dependency-free offline 'hashing trick' fallback (character-trigram feature hashing) added specifically to allow genuine end-to-end verification in network-restricted environments (CI, offline demos, this project's own development sandbox) without ever depending on a mocked embedding call.

Neither BGE nor OpenAI could be reached from this project's development sandbox (no path to huggingface.co or api.openai.com), which is a sandbox network-policy fact, not a code limitation - both providers are complete, real production code meant to be verified on a machine with normal internet access.

7. Hybrid Search

Vector search (Qdrant, cosine similarity) and keyword search (BM25 - either a real Elasticsearch cluster or a fully-tested in-process BM25 Okapi implementation) are combined via Reciprocal Rank Fusion (RRF) rather than a weighted score blend. RRF was chosen specifically because it only considers each candidate's rank position in each ranker's result list, not the raw score value - which sidesteps the problem of blending two incomparable score distributions (bounded cosine similarity vs. unbounded, corpus-statistics-dependent BM25 scores) that a fixed-weight blend would otherwise require re-tuning per corpus.

A genuine correctness gap in the underlying rank_bm25 library was found and fixed while building this: its IDF formula produces exactly zero for any term appearing in precisely half of a small corpus, silently zeroing that term's contribution to every score - a real risk for a legal knowledge base early in its life with only a handful of indexed documents. Fixed by flooring IDF at a small positive epsilon, the same kind of smoothing production search engines like Elasticsearch already apply.

8. MMR (Maximal Marginal Relevance)

Before re-ranking, the wide hybrid-search candidate pool is diversified using MMR: iteratively selecting the next chunk that is both relevant to the query and dissimilar to what has already been selected, controlled by a lambda parameter (0.5 by default - balanced between pure relevance and pure diversity). This directly addresses a real pattern in legal documents, where a judgment often restates its own holding in several places; without diversification, the system could return five near-paraphrases of the same sentence, wasting the LLM's context budget without adding real information.

MMR is implemented as pure vector math (no embedding-provider dependency baked into the algorithm itself), which let it be unit-tested with hand-constructed synthetic vectors independent of any real embedding model's behavior - including a direct test proving that, given one clearly most-relevant chunk, a near-duplicate of it, and a third chunk that is still relevant but genuinely distinct, MMR correctly prefers the distinct chunk once the top pick is already selected.

9. Re-ranking

Two re-ranking providers exist behind one interface: a real BAAI/bge-reranker-large cross-encoder (jointly scores query and passage together, catching relevance a bi-encoder or BM25 can miss - but needs the same HuggingFace download this sandbox cannot reach), and a dependency-free 'lightweight' reranker blending normalized upstream RRF score with query-term Jaccard overlap. The lightweight reranker is a real, defensible, fully-tested heuristic, not a placeholder - but it is explicitly documented as not a substitute for a cross-encoder's semantic judgment.

After re-ranking, chunks scoring below a configurable confidence threshold are dropped entirely rather than passed to the LLM as weak context - this is a hard gate in the pipeline (implemented in code, not merely a prompt instruction), directly serving the system's anti-hallucination goal: the LLM never sees a context chunk the retrieval pipeline itself doesn't trust.

10. Citation Generation

This is the project's central anti-hallucination mechanism. The LLM is instructed to place simple inline tags ([C1], [C2], ...) next to any claim it makes, referring to numbered context passages it was given. The citation formatter then does the opposite of what might be expected: it does not ask the model what page or document it used. It scans the model's answer text for these tags and looks up the corresponding page number and document name from the retrieval pipeline's own trusted data - the same page numbers Milestone 1's parser extracted directly from the PDF. The model's only real job is to place a tag roughly correctly; it is never given the opportunity to invent a page number, because it is never asked for one.

This guarantee was proven directly with a test where the model's answer prose incorrectly claims 'page 999' while the correctly-cited structured citation still reports the true page, because the citation was never sourced from the model's own text in the first place. Two further guarantees are enforced in code, not just prompted for: if retrieval returns zero chunks, the system returns the exact mandated no-evidence phrase without calling the LLM at all (deterministic, free, and immune to the model 'helpfully' guessing); and if the model answers without a single citation tag at all, the system marks the answer as not having sufficient evidence rather than presenting an uncited claim as trustworthy.

11. Frontend

A real Next.js 15 App Router application with TypeScript and Tailwind CSS v4, built around a deliberate design system rather than a generic chat-app template: a dark 'legal workbench' palette (brass, forensic ledger green, warm parchment text) and a signature 'Exhibit Ledger' panel where citations appear as numbered tabs echoing physical exhibit dividers used in litigation binders - the numbering is literal (first-citation order in the answer), not decorative.

The query console streams answers token-by-token over Server-Sent Events, rendering inline [Cx] tags as clickable citation chips as they arrive; clicking a chip or an exhibit tab links the two panels together. shadcn-style UI primitives (Radix primitives + class-variance-authority) were hand-authored rather than pulled from shadcn's CLI registry, since that registry's own site was unreachable from the sandbox - functionally identical output, since shadcn components are vendored source you own regardless of how they're generated.

12. Backend

FastAPI, organized as: app/core (config, logging, rate limiting), app/schemas (Pydantic contracts for every stage), app/services (the actual business logic - parser, chunker, embeddings, vector store, keyword search, hybrid search, MMR, reranker, retriever, LLM providers, prompts, citation formatter, QA orchestration), and app/api/routes (thin HTTP layer). Every service is built with constructor-injected dependencies specifically so it can be tested with real logic and fake/offline externalities (a deterministic fake embedder, an in-process BM25 index, a canned-but-content-aware LLM stand-in) rather than needing real network access or paid API calls to prove correctness.

A production-hardening pass added real, tested rate-limiting middleware (in-memory sliding window, per-client-IP, with a documented Redis upgrade path for horizontal scaling) and token-usage tracking on every LLM call (input/output token counts captured directly from each provider's API response, feeding both cost visibility and the evaluation harness's token-usage metric).

13. Deployment

Multi-stage production Dockerfiles for both services (non-root users, health checks, gunicorn+uvicorn workers on the backend with configuration in a dedicated gunicorn_conf.py, Next.js standalone output on the frontend), a complete docker-compose.yml (backend, frontend, Qdrant, Elasticsearch - deliberately no Postgres, since nothing in the application uses a relational database), deployment configs for Render (render.yaml), Railway (per-service railway.json files), and Vercel (frontend only - Vercel's serverless model is a poor fit for a long-running FastAPI process with persistent search-engine connections), and a five-job GitHub Actions CI pipeline (backend tests, frontend build, lint, Docker build validation, deployment-readiness config validation).

See DEPLOYMENT.md for the complete guide (SSL, secrets management, logging, monitoring, scaling, backups, cost estimates) and an explicit verification checklist distinguishing what was actually confirmed in the development sandbox from what requires real infrastructure (a Docker daemon, real cloud accounts) this sandbox does not have.

14. Limitations

This project's development sandbox has no Docker daemon/CLI at all, and its network allow-list excludes Docker Hub, Elastic's container registry, HuggingFace, api.openai.com, and Google's Gemini API - meaning several real, complete pieces of production code could not be exercised end-to-end in this specific environment: the BAAI/bge-large-en-v1.5 and OpenAI embedding providers, the BAAI/bge-reranker-large cross-encoder, a live Elasticsearch cluster, the OpenAI and Gemini LLM providers, and the actual `docker build`/`docker compose up` process for both services.

Where a real external dependency could not be reached, this project consistently built a genuine, tested, offline-capable alternative (an in-process BM25 implementation, a dependency-free 'lightweight' reranker, a deterministic hashing-trick embedding fallback) rather than mocking the gap away - and every such substitution is explicitly documented in the code and READMEs, including in the specific places where it affects interpretation of a result (most notably the evaluation run in Evaluation_Report.pdf, which used a stand-in LLM and should not be read as a verdict on real answer quality).

api.anthropic.com specifically IS reachable from this sandbox (confirmed: a request returns 404, not a network-blocked 403) - the only missing piece for real Claude-generated answers is a configured ANTHROPIC_API_KEY, which was not available in this environment. This is a credentials gap, not a network gap, and is the most straightforward of these limitations to close.

The evaluation Golden Set (18 questions) is intentionally smaller than the 100-question target mentioned in the original assignment brief, scoped to what could be hand-verified against the one real sample document available, with the dataset schema and CLI harness built to scale to 100+ questions with zero code changes (see Approach Document's Future Improvements and the evaluation README section).

15. Future Improvements

Verify all sandbox-network-blocked paths (BGE/OpenAI embeddings, BGE cross-encoder reranking, a live Elasticsearch cluster, OpenAI/Gemini LLM calls, Docker image builds) on infrastructure with normal internet access and a Docker daemon - this is the single highest-priority next step, since the code for all of these already

exists and is believed correct but is unconfirmed in practice.

Scale the Golden Set from 18 to 100+ questions across a larger, more varied real document corpus (multiple Acts, Judgments, Tax Documents, and POV Documents rather than one sample judgment), which the existing dataset schema and CLI already support without code changes.

Add a real LLM-judge pass (using Claude or another model) alongside the current deterministic faithfulness/answer-correctness proxies, to more directly approximate RAGAS-style semantic evaluation rather than token-overlap heuristics.

Move rate limiting from in-memory per-process counters to Redis-backed shared counters before running more than one backend replica, and add a query-result cache (also Redis-backed) for repeated or near-duplicate questions, which a legal research tool plausibly receives often.

Add real observability (Sentry error tracking, Prometheus metrics, structured LLM cost logging built on the token-usage tracking already implemented) - concretely scoped in DEPLOYMENT.md but not implemented in code, since it was out of scope for this assignment.